

ESPECIFICACIÓN

Auditoría de bootstrap agéntico — qué está cableado y qué depende de que alguien se acuerde

Fecha: 17-09-2026 · **Commit:** `fc8a32b` (rama `claude/ecstatic-ptolemy-f7cb4m`) · **Alcance:** el REPOSITORIO, no el producto **Marco:** skill `agentic-repo-bootstrap-v2` · **Verificación previa:** los cuatro pasos de `CLAUDE.md`, los cuatro en verde

Hay dos preguntas distintas sobre un repositorio y conviene no mezclarlas:

- ¿El producto funciona? Sí, y está medido: `tsc` limpio, build verde con los hashes del SBOM validados, cero duplicados de nivel módulo y **114 de 114 casos PASA**. Esa pregunta está contestada y no es de lo que trata este documento.
- ¿El repositorio se puede TRABAJAR? Es otra pregunta, y tiene otra respuesta. NEX tiene una disciplina de ingeniería poco común —114 casos que prueban invariantes de negocio reales, tres auditores propios, specs versionadas con su PDF— y **casi nada de esa disciplina está cableado**. Todo depende de que la próxima sesión lea 229 KB de `CLAUDE.md` y se acuerde de correr cuatro comandos que nadie impone.

Cómo leer esto. Ningún hallazgo de acá abajo es un defecto del producto ni una crítica al criterio con que se escribió `CLAUDE.md`: ese documento es la razón de que el proyecto haya podido crecer a 25.922 líneas sin perder coherencia, y su contenido es memoria real, bien escrita y ganada con incidentes concretos. Lo que este documento sostiene es que **ese contenido está en la casa equivocada**: una regla, el porqué de la regla y la fecha en que se descubrió son tres cosas con tres vidas útiles distintas, y hoy viven en el mismo párrafo y se cargan las tres en cada sesión. Todos los conteos son reproducibles: los comandos están en §10.

0. Cómo usar este documento

Es **autocontenido**: se puede abrir en una sesión nueva sin más contexto que el repositorio. Cada hallazgo trae **qué se midió · el número · por qué importa · qué lo arreglaría**. No propone tocar el producto: todo lo que sigue es infraestructura de trabajo.

El marco es el skill `agentic-repo-bootstrap-v2`, que existe para que un repositorio sobreviva a muchas sesiones de agentes sin perder contexto ni derivar de sus decisiones. Sus seis objetivos son memoria con una sola fuente de verdad, guardrails deterministas, gates como tests, ceremonia calibrada, doctrina de despacho y un git legible en paralelo. Este documento recorre los seis contra lo que hay hoy.

1. La verificación, corrida antes de opinar

Los cuatro pasos que `CLAUDE.md` declara obligatorios tras cada edición, corridos el 17-09-2026:

PASO	COMANDO	RESULTADO
1 · Tipos	<code>npx tsc --jsx preserve --allowJs --noEmit --skipLibCheck pipeline_comercial.jsx</code>	verde, sin errores TS1
2 · Duplicados	<code>grep</code> de símbolos de nivel módulo, <code>sort + uniq -d</code>	vacío
3 · Build	<code>node build_app.mjs</code>	verde · 40,6 MB
4 · Suite	<code>PLAYWRIGHT_BROWSERS_PATH=/opt/pw-browsers node run_tests.mjs</code>	114/114 PASA , 0 errores de página

Los cuatro pasan y los cuatro son voluntarios. No hay CI, no hay hooks, no hay `.claude/settings.json`. La única cosa que los ejecuta es que alguien se acuerde. En 108 commits eso funcionó; es una apuesta a la memoria de quien edita, y es la apuesta que el resto del documento propone retirar.

2. CLAUDE.md — el hallazgo central

2.1 Pesa 57.000 tokens y se carga entera, siempre

MEDICIÓN	VALOR
Líneas	425
Bytes	229,306
Tokens aproximados (bytes ÷ 4)	~57.000
Bytes por línea (promedio)	540
Límite del skill	150 líneas

El problema no es el conteo de líneas —425 contra 150 no sería grave— sino que **son líneas enormes**. Un CLAUDE.md se carga en el contexto de CADA sesión, y la precisión de un agente se degrada a medida que crece lo que siempre está cargado. Hoy toda sesión de NEX arranca pagando 57.000 tokens antes de leer una línea de código, tenga que tocar el motor de líneas o corregir un rótulo.

Reparto por sección:

SECCIÓN	BYTES	%
Reglas de dominio (invariantes)	172.679	75,3 %
Arquitectura y build	22.404	9,8 %
Documentación del proyecto	20.136	8,8 %
Convenciones del código	11.556	5,0 %
Contrato con el servidor	1.483	0,6 %
Flujo de trabajo con el usuario	647	0,3 %

La regla 31 sola pesa 26.185 bytes (~6.500 tokens) —más que las cinco secciones no-dominio juntas— y su propio texto la describe como «BLOQUE DESECHABLE». Una sesión que corrija un color en el Kanban carga íntegro el diseño del modo Directorio.

2.2 Tres cosas distintas en el mismo párrafo

Las 59 reglas de dominio mezclan, sin separador, tres géneros con tres vidas útiles:

GÉNERO	EJEMPLO TEXTUAL	VIDA ÚTIL	CASA QUE LE DA EL SKILL
La regla	«Curse nunca por email: el cliente acepta SOLO firmando en el portal»	permanente	vault/conocimiento/invariantes.md , con columna dónde se verifica
El porqué	«se cortó en el gate y no limpiando banderas una por una, porque olvidar una dejaría girar sin aceptación vigente»	permanente, inmutable	un ADR numerado
La historia	«el primer intento tenía el motor correcto y el badge descartaba al reemplazante antes de preguntar»	se consulta una vez	el log de sesión

Conteo del género histórico dentro de CLAUDE.md : 35 «pedido del usuario», 62 «antes», 35 «medido», 13 «se retiró», 6 «primer intento», y 14 fechas distintas citadas entre el 02-09 y el 16-09-2026. Es un log de sesión excelente ocupando el lugar de un archivo de reglas.

La consecuencia visible es la numeración, que ya no se puede ordenar: 13-octies-bis , 15-bis-bis , 15-quater-bis , 30-ter aparece antes que 24 , y 17 está al final. Ese es el síntoma de un documento append-only al que se le pide ser un índice: cada corrección nueva no tiene dónde ir salvo colgando de la anterior. La numeración no se arregla renumerando —se arregla cuando la historia deja de vivir ahí.

2.3 Ya se desfasó, y no hay nada que lo detecte

CLAUDE.MD AFIRMA	MEDIDO EL 17-09-2026	DESVÍO
«~21.000 líneas»	25.922	+23 %
«118 componentes»	154 funciones con inicial mayúscula	+31 %
«genera ~31 MB»	40,6 MB	+31 %

Ninguna de las tres tiene gate, así que las tres se desfasaron en silencio. Es la lección número uno del skill: **dos fuentes de la misma verdad se contradicen, siempre, y sin avisar**. El propio repositorio ya sabe esto —lo dice en la regla 9-bis sobre los umbrales duplicados y en la regla 28

sobre el rótulo de etapa— pero la lección no se aplicó al documento que enuncia las reglas.

3. Doce invariantes declarados, cuatro verificados

El fuente declara `INVARIANTES` (línea 12.764) con doce códigos y su autoridad: el contrato que el resolver GraphQL debe implementar 1:1. Buscados uno por uno en `tests_asignacion_lineas.js`:

INVARIANTE	MENCIONES EN LA SUITE	ESTADO
OTG-02	1	cubierto
VER-01	2	cubierto
GIR-02	2	cubierto
TEN-01, RAT-01, IDM-01, LIN-01, OTG-01, GIR-01, ATR-01, CRY-01, PRI-01	0	sin gate

Ocho de doce no se verifican en ninguna parte. Entre ellos ATR-01 (la atribución de tasa, que es el control sobre el precio que sale por la puerta) y LIN-01. El skill pide exactamente una tabla *invariante / dónde se verifica* y trata la columna vacía como el hallazgo: un invariante sin gate es una aspiración. Los cuatro cubiertos demuestran que acá el patrón ya se sabe hacer —lo que falta es completarlo, no inventarlo.

Matiz que el propio `CLAUDE.md` deja escrito y hay que respetar: **esto no es un control de seguridad**, porque el atacante ES el cliente. El gate no reemplaza al resolver; lo que hace es impedir que la anticipación del cliente se separe del contrato sin que nadie lo note.

4. Los gates ya existen — y ninguno está cableado

El repositorio contiene **48 KB de herramienta de auditoría propia**, de buena calidad, que nadie ejecuta automáticamente:

HERRAMIENTA	QUÉ MIDE	QUIÉN LA CORRE
<code>auditar_muerto.mjs</code>	símbolos sin referencias con transitividad, <code>useState</code> muertos, props no declaradas, CSS en los dos sentidos	a mano, cuando alguien se acuerda
<code>auditar_aislamiento.mjs</code>	qué funciones leen estado mutable, propagado por el grafo de llamadas	a mano
<code>regresion_diferencial.mjs</code>	regresión diferencial entre versiones	a mano
<code>tests_asignacion_lineas.js</code>	114 casos de invariantes de negocio	a mano

Lo que falta no es la herramienta: es el cable.

MECANISMO DEL SKILL	ESTADO EN NEX
<code>.claude/settings.json</code> con hooks	no existe (<code>.claude/</code> sólo tiene <code>launch.json</code> y una copia del skill <code>datamart-ui</code>)
<code>protect_paths.sh</code> — ADRs, specs entregadas, <code>vendor/</code>	no existe
<code>gitflow_guard.sh</code> — integración con <code>--no-ff</code> , sin commits directos	no existe
CI con el quartet de gates	no existe
Tags de release	0 tags en 108 commits
<code>tests/</code> como capa	no existe; la suite es un archivo suelto de 3.370 líneas en la raíz
Vault y tablero de estado	no existe; el estado del proyecto está disperso en 14 fechas dentro de <code>CLAUDE.md</code>

Dos observaciones con evidencia propia del repositorio sobre por qué esto importa acá más que en un proyecto normal:

- `CLAUDE.md` ya documenta cuatro clases de defecto que ninguno de los cuatro pasos detecta: la colisión entre parámetro y variable local (12-09), el bloque declarado antes de su dependencia (13-septies), el componente no importado (30-ter) y la poda que se llevó `CUENTAS_DEMO`. Las cuatro veces la conclusión escrita fue la misma —«lo que verifica un cambio en el detalle es abrir el detalle»— y las cuatro veces ese paso quedó como instrucción, no como gate.
- La suite ya tiene el hábito de la sonda negativa: 16 menciones a casos que deben fallar. El skill considera ese hábito el más importante de la capa de contrato. Está presente y no está nombrado como política, así que depende de que cada caso nuevo lo repita por imitación.

5. Datos: los RUT son sintéticos, las razones sociales no

Medido sobre los primeros 4 MB de `datos_inyectados.js` :

CAMPO	RANGO	DISTINTOS	VEREDICTO
<code>RUTEmisor</code> (clientes)	1.226.447 – 98.698.635	500	sintético — fuera del rango real de empresa (60M–79M)
<code>RUTRecep</code> (deudores)	1.219.340 – 98.849.202	555 en la muestra	sintético
<code>RznSocRecep</code> (deudores)	—	465 en la muestra	reales y reconocibles

Las razones sociales de la muestra incluyen Codelco, Falabella, Cencosud, CMPC, Sodimac, Escondida (BHP), Antofagasta Minerals, Coca Cola Embonor, Agrosuper, Abbott Laboratories Chile, Adidas Chile y el **Ministerio de Obras Públicas**.

O sea: un HTML de 40,6 MB que sale del edificio **atribuye facturas, montos, moras y notas de riesgo inventadas a empresas reales nombradas**, algunas de ellas organismos del Estado. Los identificadores son sintéticos, que es lo que hace que no sea una filtración; los nombres no lo son, que es lo que hace que una captura de pantalla afirme algo sobre un tercero identificable.

Esto es **plausiblemente deliberado** —un demo para BICE y Factoring Security con «Empresa 47» en vez de Codelco no se ve como el producto— y no es un defecto mientras sea una decisión. El hallazgo es que hoy **no está decidido en ninguna parte**: no hay ADR, no hay gate, y el criterio vive sólo en el generador. Lo que el skill pide acá es barato: una decisión escrita (ADR) más un gate `no-real-data` que la haga cumplir o la obligue a re-litigarse a propósito. La regla de la casa ya apunta en esa dirección —«Datos sintéticos DETERMINISTAS», «Nada de Math.random»— pero cubre el determinismo, no la identificabilidad.

6. Un artefacto generado, versionado y con un mes de deriva

`pipeline.zip` (2,8 MB versionados) contiene un build de `pipeline_comercial.html` de 29.634.808 bytes, fechado el 12-08-2026. El build de hoy pesa 40,6 MB. Es una segunda copia de un artefacto regenerable, con un mes y 11 MB de deriva, que nadie va a notar porque nada la compara.

El `.gitignore` ya aplica el criterio correcto a los otros dos casos —ignora `pipeline_comercial.html` y los PNG de las capturas, con el razonamiento escrito al lado— así que esto es un olvido, no un desacuerdo. `babel.min.js.descarga` (2,8 MB) y `saved_resource` (407 KB) son dependencias vendorizadas y sí corresponde versionarlas: están fijadas por hash y el build las verifica.

7. Dos funciones son el 22 % del fuente

SÍMBOLO	LÍNEAS	NOTA
<code>PipelineComercial</code>	2.905	componente raíz, 50 <code>useState</code>
<code>DealDrawer</code>	2.794	el detalle de la oportunidad
Juntos	5.699 de 25.922	22,0 %

No es urgente y no rompe nada: es el techo práctico de lo que un agente puede editar sin releer miles de líneas, y explica por qué las ediciones quirúrgicas con anclas únicas son la política de la casa. Vale anotarlo como deuda medida, no como tarea. El archivo tiene además 940 símbolos de nivel módulo y 258 constantes en mayúsculas, lo que indica que el problema no es la modularidad general —es buena— sino estos dos componentes concretos.

8. Qué propondría el bootstrap, calibrado a este repo

El skill trae una **escalera de ceremonia** justamente para no cobrar el paquete completo donde no corresponde. Mapeada a NEX:

NIVEL	QUÉ ES ACÁ	QUÉ LLEVA
T1	un motor (<code>asignarLineas</code> , <code>verifDecision</code> , <code>asignarGiros</code>), un invariante nuevo, un contrato con el servidor	feature + spec + plan + tareas + ADR + rama + log
T2	una regla nueva del catálogo de otorgamiento, una fila de configuración del tenant, un rótulo	rama corta + un commit verde + fila del tablero
T3	tipos, anotaciones, el tablero	commit directo

Buena parte de lo que hoy entra a `CLAUDE.md` como regla numerada es **T2 sobre un mecanismo ya gateado** y no necesita quedar en el documento que se carga siempre.

El orden que tendría sentido, de mayor a menor retorno por riesgo:

1. **Partir** `CLAUDE.md` (hallazgos 2.1–2.3). El documento queda en ~140 líneas con comandos, gates, reglas núcleo y mapa. Las 59 reglas **no se borran**: se reubican en `vault/conocimiento/invariantes.md` (la regla, con su columna *dónde se verifica*), ADRs (el porqué, inmutable) y logs de sesión (la historia). Es cirugía sobre el documento más valioso del repo: el trabajo es reubicar sin perder un párrafo, **no resumir**.
2. **Cablear lo que ya existe** (hallazgo 4): los tres auditores y la suite como `tests/contract/`, más hooks deterministas y un CI con los cuatro pasos de §1.
3. **Cerrar la tabla de invariantes** (hallazgo 3): un gate por cada uno de los ocho sin cobertura, cada uno con su sonda negativa.
4. **Decidir los datos** (hallazgo 5): un ADR que diga qué se hace con las razones sociales reales, y el gate que lo sostenga.
5. **Sacar** `pipeline.zip` del versionado (hallazgo 6).
6. **Anotar** los dos componentes grandes como deuda medida (hallazgo 7). No tocar.

9. Lo que NO hay que hacer

- **No resumir** `CLAUDE.md`. El contenido está ganado con incidentes reales y cada párrafo explica por qué una regla es como es. Resumir pierde exactamente lo que lo hace valioso; el movimiento correcto es reubicar por vida útil.
- **No reenumerar las reglas** como arreglo. `13-octies-bis` es un síntoma, no la enfermedad.
- **No tocar el producto**. Los 114 casos pasan; nada de este documento pide cambiar comportamiento.
- **No versionar dos veces la misma verdad**. Es el error que este documento denuncia: si un conteo (líneas, componentes, MB) va a aparecer en la documentación, que lo produzca un gate.
- **No introducir worktrees ni herramientas opcionales** para cerrar esto. El skill las considera mejoras, no requisitos, y un bootstrap que se bloquea en tooling opcional falla por la razón equivocada.

10. Cómo reproducir cada número

Todos los conteos de este documento salen de comandos sobre el repositorio, sin herramientas nuevas. Las cifras corresponden al 17-09-2026, commit `fc8a32b`; un conteo sin fecha no es una medición.

```

# §1 - la verificación completa
npx tsc --jsx preserve --allowJs --noEmit --skipLibCheck pipeline_comercial.jsx
grep -oE "^(function|const|let|var) [A-Za-z0-9_]+" pipeline_comercial.jsx | awk '{print $2}' | sort | uniq -d
node build_app.mjs
PLAYWRIGHT_BROWSERS_PATH=/opt/pw-browsers node run_tests.mjs

# §2.1 - peso de CLAUDE.md y reparto por sección
wc -l -c CLAUDE.md
awk '/^## /{if(h)printf "%7d B  %s\n", b, h; h=$0; b=0; next} {b+=length($0)+1} END{if(h)printf "%7d B  %s\n", b, h}' CLAUDE.md

# §2.2 - el género histórico dentro de las reglas
for p in "pedido del usuario" "antes" "medido" "se retir" "primer intento"; do
  printf "%5d  %s\n" "$(grep -oiF "$p" CLAUDE.md | wc -l)" "$p"; done
grep -oE "[0-9]{2}-09(-2026)?" CLAUDE.md | sort -u

# §2.3 - lo que CLAUDE.md afirma contra lo medido
wc -l pipeline_comercial.jsx
grep -oE "^(function [A-Z][A-Za-z0-9_]*|const [A-Z][A-Za-z0-9_]* = \()" pipeline_comercial.jsx \
  | grep -oE "[A-Z][A-Za-z0-9_]*" | sort -u | wc -l

# §3 - cobertura de los invariantes declarados
for c in TEN-01 RAT-01 IDM-01 LIN-01 OTG-01 OTG-02 VER-01 GIR-01 GIR-02 ATR-01 CRY-01 PRI-01; do
  printf "%-8s %s\n" "$c" "$(grep -c "$c" tests_asignacion_lineas.js)"; done

# §5 - rango de los RUT y razones sociales
node -e 'const s=require("fs").readFileSync("datos_inyectados.js","utf8").slice(0,4e6);
const r=[...s.matchAll(/"RUTEmisor":"([0-9]+)-/g)].map(m=>+m[1]);
console.log(Math.min(...r), Math.max(...r), new Set(r).size);'

# §6 - el artefacto rancio
unzip -l pipeline.zip
git log -1 --format="%h %ad" --date=short -- pipeline.zip

# §7 - tamaño de los dos componentes grandes
awk 'NR>=23019{n++} END{print "PipelineComercial:", n}' pipeline_comercial.jsx
awk 'NR>=23019' pipeline_comercial.jsx | grep -c "useState("

```